

Lecture 04

In this Session, we will be examining the third control structure - Looping/Iteration/Repetition

Looping/Iteration/Repetition

This provides a way to repeat a statement or block of statements zero or more times. Thus

- are you now able to write a program that might have been logistically impossible to write before such as adding the numbers up to 1_000_000
- execute a code block an indeterminate number of times

Contents:

There are two kinds of loops:

1. [while](#)
2. [for](#)
 - [Using the range object](#)
 - [Using an enumerable](#)
 - [Using the enumerate function](#)
3. [Alignment](#)
4. [Using the continue and break keyword](#)
5. [Nested Loop](#)
6. [Summary](#)

while

It is easier to create indeterminate loops with this loop. i.e. a loop that is not constrained by a range object, an iterable or an enumeration.

You have to use the `break` statement to terminate such loops.

Just like how you have indentation to indicate semantic blocks, the same is true for loop bodies.

Unlike C#, python does not have do-while loops

Syntax for while statement

```
while «loop_condition»:  
    statements  
    «updater statement»
```

If the update statement is missing, then it will be an infinite loop (non terminating loop)

```

#while loop
counter, end = 0, 100                                #loop initializer
while counter < end:                                  #loop condition
    print(f'{counter}', end=' ')
    counter += 1                                       #updater statement
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80
81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99
# the last statement in the following code snippet is not part of the loop

counter, end = 8, 10
while counter < end:
    print(f'counter: {counter}')
    counter += 1
print('This statement is not part of the while loop')

# change the value of counter to 11
# and run the code again
counter: 8
counter: 9
This statement is not part of the while loop
#while example
# updater counts down
start = 5
stop = 0
while start > stop:
    print(f'{start}', end=' ')
    start -= 1
5 4 3 2 1
end = 3
sum = 0
counter = 0
while counter < end:
    sum += int(input('Enter a number: '))
    counter += 1

print(f'The sum of the {end} numbers is {sum}')
The sum of the 3 numbers is 17
# this program does not use a counter to terminate the loop
# it uses a condition based on the sum of the inputs
target = 20
sum = 0
counter = 0
while sum < target:
    sum += int(input('Enter a number: '))
    counter += 1

print(f'It took {counter} number of additions to reach The sum of {sum}')
It took 6 number of additions to reach The sum of 23
#while example using the and keyword
start = 1
stop = 100

```

```

sum = 0
while start < stop:
    if start % 5 == 0 and start % 2 != 0:           #combining two conditions
        with the and keyword
            sum += start
            start += 1

print(f'The sum of the non-even multiples of 5 less than 100 to {sum}')
The sum of the non-even multiples of 5 less than 100 to 500
#while example (3)
start = 1_000_000
stop = 2_000_000
sum = 0
while start < stop:
    if start % 3 == 0:
        sum += start
        start += 1

print(f'The sum of the multiples of 3 from 1,000,000 to {stop:,.0f} is
{sum:,.0f}')
# why can't you use the variable start in the print statement?
The sum of the multiples of 3 from 1,000,000 to 2,000,000 is 499,999,500,000
#an example that uses a flag
to_go = True
total = 0

while to_go:
    x = int(input('Enter a number: '))

    if x < 0:
        to_go = False    # signal the loop should end AFTER this iteration

    # logic that must still run even when stopping
    print(f'You entered: {x}')

    if x > 0:
        total += x

print('Done! Total of positive numbers:', total)
You entered: 6
You entered: 4
You entered: 3
You entered: 4
You entered: 2
You entered: 2
You entered: 4
You entered: 5
You entered: 6
You entered: 6
You entered: -1
Done! Total of positive numbers: 42

```

for loop

The for loop is a **determinate loop** i.e. the number of iterations is determined by an iterable such as a string, list, tuple, set, dictionary or range() object.

Syntax for the for statement

```
for iterator_variable in range_object:  
    «Body of loop»
```

Using the range object

range([start = 0], end, [step])

n.b. The arguments to the range constructor may only be integers

This works really well with the for loop

```
#for example  
for _ in range(4):      #the underscore is used to quiet the linter  
    print('Hello world')  
Hello world  
Hello world  
Hello world  
Hello world  
#for example  
for x in range(10):      #the value for the range function represent the end  
    value  
    print(f'x: {x}')  
x: 0  
x: 1  
x: 2  
x: 3  
x: 4  
x: 5  
x: 6  
x: 7  
x: 8  
x: 9  
#for example (2)  
for x in range(10, 12):  #the two values represent the start and end  
    print(f'x: {x}')  
x: 10  
x: 11  
#for example (3)  
for x in range(10, 30, 5):  #the three values represent the start, the end  
    and the step  
    print(f'x: {x}')  
x: 10  
x: 15  
x: 20  
x: 25
```

```

#for example (3)
for x in range(30, 10, -3):      #a sequence that decreases
    print(f'x: {x}')
x: 30
x: 27
x: 24
x: 21
x: 18
x: 15
x: 12
#for example (3)
for x in range(0, -20, -6):      #the three values represent the start, the
    print(f'x: {x}')            end and the step
x: 0
x: -6
x: -12
x: -18

```

Using an iterable object

Syntax for iterable statement

This works well with iterables (sequences) like lists, string, tuples etc.

```

for «iterator variable» in «iterables»:
    «Body of loop»
#for example (4)
a = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
for x in a:
    print(f'{x}', end=', ') #end='' prevents the print function from adding
a new line
2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
#nested for loop
for let in 'ABC':
    for num in [1, 2, 3, 4, 5]:
        print(f'{let}{num}', end=' ')
    print()
A1 A2 A3 A4 A5
B1 B2 B3 B4 B5
C1 C2 C3 C4 C5

```

Using the enumerate function

for syntax with enumerate() (get index and value)

```

for «index», «value» in enumerate«enumerable»:
    «Body of loop»

```

The enumerate() function adds an index to an iterable, returning both the **position** and the **value**. It works with any iterables such as list, string, tuple or dictionary

```

a = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
#for example (4)

```

```

for index, value in enumerate(a):
    print(f'{index}: {value}')
0: 2
1: 3
2: 5
3: 7
4: 11
5: 13
6: 17
7: 19
8: 23
9: 29
b = {'one': 1, 'two': 2, 'three': 3, 'four': 4, 'five': 5}
#for example (5)
for key, value in b.items():
    print(f'{key}: {value}')
one: 1
two: 2
three: 3
four: 4
five: 5

```

Alignment when printing tables

When printing tabular data, it looks professional if the columns are aligned.

- right alignment
`>` right aligns the current content to the specified column
- left alignment
`<` left aligns the current content to the specified column
- center alignment
`^` aligns the current to the specified column

#alignment/formatting

```

for x in range(1, 11):
    print(f'{x:>3}{x**2:>7}{x**3:>9}{x**4:>11}')    #> right aligns < left

```

aligns

1	1	1	1
2	4	8	16
3	9	27	81
4	16	64	256
5	25	125	625
6	36	216	1296
7	49	343	2401
8	64	512	4096
9	81	729	6561
10	100	1000	10000

*# Separators are use to improve readability of large integral numbers
 # There are only two separators that are commonly used in formatting
 numbers: the comma and the underscore.*

```

for x in range(10, 101, 10):
    square = f'{x ** 2:}_}'      # separator is underscore
    cube = f'{x ** 3:,:}'       # separator is comma
    print(f'{x:>3}{square:>8}{cube:>11}')
10      100      1,000
20      400      8,000
30      900     27,000
40     1_600     64,000
50     2_500    125,000
60     3_600    216,000
70     4_900    343,000
80     6_400    512,000
90     8_100    729,000
100    10_000   1,000,000
# using different fillers
# for easier explanation, the single print statement broken into multiple
lines
for x in range(1, 11):
    print(f'{x:_>3}', end='')      # spacer is underscore
    print(f'{x**2:~>7}', end='')   # spacer is plus
    print(f'{x**3:=>9}', end='')   # spacer is equal
    print(f'{x**4:>11}')           # spacer is default (space)
_1+++++1=====1                  1
_2+++++4=====8                  16
_3+++++9=====27                 81
_4+++++16=====64                256
_5+++++25=====125               625
_6+++++36=====216              1296
_7+++++49=====343              2401
_8+++++64=====512              4096
_9+++++81=====729              6561
_10+++100=====1000             10000
#more formatting
# using width and precision as variables

header = '      x      x^2      x^3      x^4'
print(header)
print(f'  {'=' * len(header)}')      #len()
function give the number of letters in the header
width = 10
precision = 6
for raw in range(1, 11):
    x = raw / 40

print(f'{x:>{width}.{precision}f}{x**2:>{width}.{precision}f}{x**3:>{width}.{
precision}f}{x**4:>{width}.{precision}f}')      #> right aligns < left aligns
      x      x^2      x^3      x^4
=====
0.025000  0.000625  0.000016  0.000000
0.050000  0.002500  0.000125  0.000006
0.075000  0.005625  0.000422  0.000032
0.100000  0.010000  0.001000  0.000100

```

```

0.125000  0.015625  0.001953  0.000244
0.150000  0.022500  0.003375  0.000506
0.175000  0.030625  0.005359  0.000938
0.200000  0.040000  0.008000  0.001600
0.225000  0.050625  0.011391  0.002563
0.250000  0.062500  0.015625  0.003906
#more formatting
# formatting sub-parts

print('Length      Weigth      Cost')
print('=====')
for length in range(5, 41, 5):
    weight = f'{length * 3.1415:.3f}'           #can also combine
alignment with decimal places
    cost = f'${length * 3.1415 * 1250:,.2f}'    #there is no easy way to
do currency
    print(f'{length:>4}{weight:>13}{cost:>16}')  #> right aligns < left
aligns
Length      Weigth      Cost
=====
     5      15.708      $19,634.38
    10      31.415      $39,268.75
    15      47.123      $58,903.12
    20      62.830      $78,537.50
    25      78.538      $98,171.88
    30      94.245     $117,806.25
    35     109.953     $137,440.62
    40     125.660     $157,075.00
# even more formatting
# using octal and hexadecimal
# b -> binary
# c -> character
# o -> octal
# x -> hexadecimal
# X -> hexadecimal (uppercased letter)
# The above ONLY works with integer values

import random

print('ASCII  Octal  Hex  Letter  chr()  Random value')
print('=====')
for x in range(97, 123):
    w = (0.5 - random.random()) * 50           # produces a random float between
-25 and 25
    u = f'{w:+.4f}'                             # the plus character forces the
display of the sign of the value
    print(f'{x:>4}', end='')                     #> right aligns to the fourth
column
    print(f'{x:>8o}', end='')                     #> right aligns to the eighth
column formatted as octal
    print(f'{x:>7X}', end='')                     #> right aligns to the seventh
column formatted as uppercased Hexadecimal

```

```

    print(f'{x:>8c}', end='')           #> right aligns to the eight
column formatted as a character
    print(f'{chr(x):^13}', end='')      #^ center aligns to the thirteenth
column
    print(f'{u:>8}')                     #> right aligns to the eight
column

```

The above way of printing is *EXTREMELY* inefficient, it was done this way to explain the formatting of each value

e.g. `print(f'{x:>4}{x:>8o}{x:>7X}{x:>8c}{chr(x):^13}{u:>8}')`

ASCII	Octal	Hex	Letter	chr()	Random value
97	141	61	a	a	-15.2557
98	142	62	b	b	+20.7628
99	143	63	c	c	-8.1191
100	144	64	d	d	+0.7640
101	145	65	e	e	-20.1412
102	146	66	f	f	+14.2132
103	147	67	g	g	+3.2278
104	150	68	h	h	-0.8652
105	151	69	i	i	+2.0668
106	152	6A	j	j	-10.3717
107	153	6B	k	k	-17.8764
108	154	6C	l	l	-7.0407
109	155	6D	m	m	+1.6704
110	156	6E	n	n	+19.7925
111	157	6F	o	o	-5.1255
112	160	70	p	p	+1.2426
113	161	71	q	q	-0.1550
114	162	72	r	r	+21.5997
115	163	73	s	s	+7.1073
116	164	74	t	t	-22.9651
117	165	75	u	u	-7.9346
118	166	76	v	v	+1.5527
119	167	77	w	w	-10.0036
120	170	78	x	x	+0.4425
121	171	79	y	y	+14.9756
122	172	7A	z	z	+23.9048

Using the continue and break keywords

These two keywords changes the normal operation of a loop (both while and for)

The continue keyword

- When the `continue` keyword is encountered, the rest of the loop body is skipped.
- Processing resumes from the top of the loop body
- If your updater statement is after the continue keyword, then there is a possibility of a non-terminating loop

#using the continue keyword

```

sum = 0
for x in range(100):      #the value for the range funtion represent the end
    value
    if x % 5 == 0:
        continue
    sum += x

print(f'Sum of non-multiple of five less than 100: {sum}')
Sum of non-multiple of five less than 100: 4000

```

The break keyword

- When the `break` keyword is encountered, the loop is immediately terminated
- Processing continues outside of the loop body

```

#while to demonstrate the break keyword
sum = 0
while True:
    value = int(input('Enter a positive number to sum: '))
    if value < 0:
        break
    sum += value

print(f'The sum of your inputs is {sum}')
The sum of your inputs is 38

```

Summary

Key Concepts

1. **While Loops:** Best for indeterminate iterations; require explicit updater statements
2. **For Loops:** Ideal for definite iterations over ranges or collections
3. **range():** Generates integer sequences with start, stop, and step parameters
4. **enumerate():** Provides both index and value when iterating
5. **Formatting:** Use alignment specifiers (>, <, ^) for professional output
6. **Loop Control:** `continue` skips current iteration; `break` exits loop entirely
7. **Nested Loops:** Allows for complex iterations, such as working with multi-dimensional data.

Best Practices

- Use `for` loops when iteration count is known or iterating over collections
- Use `while` loops for condition-based repetition
- Always include updater statements in `while` loops to avoid infinite loops
- Use `enumerate()` when you need both position and value
- Use `items()` when iterating over dictionaries to get both keys and values
- Format output with proper alignment for readability
- Use `_` as iterator variable when the value isn't needed
- Employ `break` for early exit and `continue` to skip specific iterations

Common Pitfalls

- Forgetting updater statements in `while` loops (infinite loops)
- Placing updater after `continue` in `while` loops
- Using float values in `range()` (only integers allowed)
- Off-by-one errors with range endpoints (remember: stop is exclusive)